Ejercicios de la Unidad 2

Objetos métodos y librerías de Java

Programación · 1.º DAM y DAW · Curso 2026–27

Indicaciones comunes

• Escribe el código dentro de **public static void main(String[] args)**, con la clase habitual del programa y las importaciones necesarias. Guarda cada programa en un archivo con el mismo nombre que su clase.

• Puedes usar lo trabajado en las unidades 1 y 2: variables, constantes, tipos primitivos, operadores, conversiones y las operaciones explicadas de Point, Scanner, String, Math y Random.

• No uses if, switch, el operador ternario, bucles, arrays para resolver los ejercicios, colecciones, excepciones, métodos propios ni clases adicionales. String[] args solo forma parte del esqueleto habitual de main.

• Supón que todas las entradas cumplen las condiciones de cada ejercicio. No tienes que validarlas.

• En las pruebas, las comillas muestran los límites del texto: no se escriben en la entrada. No se exige un formato especial para los decimales. Los resultados aleatorios se comprueban por su intervalo.

• Entrega los archivos .java de los ejercicios programados, las respuestas razonadas de los ejercicios 1, 3, 9 y 11 y una evidencia de las pruebas indicadas. Añade la ficha de operaciones del ejercicio 12.

1 Referencias en un panel de posiciones

Básico · Distinguir el valor de una variable primitiva de una referencia a un objeto.

Antes de ejecutar el fragmento, escribe los cinco valores que se mostrarán, en su orden. Indica cuántos objetos Point se crean y qué variables comparten objeto al terminar. Dibuja las referencias después de cada asignación a una variable de tipo Point.

Después, ejecuta el código en un programa Java y contrasta tu predicción. Explica por qué cambiar copia no modifica plazas, pero cambiar vista.y sí modifica posicion.y.

```
int plazas = 6;
int copia = plazas;
copia = 9;
Point posicion = new Point(2, 5);
Point vista = posicion;
Point alternativa = new Point(2, 5);
vista.y = 8;
vista = alternativa;
vista.x = 7;
System.out.println(plazas);
System.out.println(copia);
System.out.println(posicion.y);
System.out.println(alternativa.x);
System.out.println(vista.y);
```

Salida prevista: cinco líneas. Las variables de referencia pueden cambiar de objeto durante la ejecución.

2 Posiciones de una instalación

Básico · Elegir constructores existentes y crear objetos independientes.

En una instalación artística, representa tres posiciones con Point. Crea inicio con el constructor sin argumentos y foco con el constructor que recibe dos enteros, usando las coordenadas (6, 4).

Crea respaldo como un objeto nuevo con las coordenadas actuales de foco. Después cambia foco.x a 9 y foco.y a 1. Muestra las coordenadas de las tres posiciones y explica por qué respaldo conserva sus valores. Identifica las tres llamadas a constructores y sus argumentos.

Resultado: inicio (0, 0), foco (9, 1) y respaldo (6, 4). No basta con asignar foco a respaldo.

3 Qué recibe y qué devuelve una llamada

Básico · Identificar receptor, argumentos, sobrecarga, retorno y métodos static.

Para cada una de las cinco llamadas del fragmento, anota: receptor o clase utilizada, nombre del método, argumentos, tipo de retorno y resultado. En println indica qué aparece en pantalla y por qué no se puede guardar su retorno.

Explica qué diferencia hay entre las dos llamadas a substring y por qué sqrt se invoca de otra manera. Predice los valores de n, inicio, finalTexto y raiz antes de ejecutar.

```
String titulo = "Planetario";
int n = titulo.length();
String inicio = titulo.substring(0, 6);
String finalTexto = titulo.substring(6);
double raiz = Math.sqrt(81.0);
System.out.println(inicio);
```

Los índices de substring empiezan en 0 y el extremo final de un rango se excluye.

A)

Las dos llamadas utilizan el mismo nombre de método (substring), pero reciben una cantidad diferente de argumentos ((0, 6) frente a (6)). Esto es un claro ejemplo de sobrecarga de métodos (overloading), que permite a una misma clase ofrecer múltiples variantes de un método adaptándose a las necesidades de los parámetros que recibe:

La versión de dos parámetros define un rango explícito (inicio y fin).

La versión de un parámetro asume que el punto final es el final absoluto de la cadena de texto.

B)

El método `sqrt` pertenece a la clase `Math` y está declarado como **`static`** (estático). Los métodos estáticos pertenecen a la clase en su conjunto y no a un objeto individual. Por esta razón, se invoca directamente anteponiendo el nombre de la clase (`Math.sqrt(...)`) en lugar de requerir la creación previa de un objeto mediante el operador `new`. Por el contrario, `length()` y `substring()` son métodos de instancia, por lo que necesitan obligatoriamente un objeto concreto (`titulo`) sobre el cual ser ejecutados.

4 Etiqueta de una pieza del museo

Básico · Consultar y transformar un String usando sus métodos.

Lee el nombre completo de una pieza del museo. El nombre, una vez limpio, tendrá al menos tres caracteres. Conserva el texto original y crea otra variable con el resultado de quitar los espacios de los extremos.

Muestra el nombre limpio en mayúsculas, su longitud, el primer carácter, el último carácter y las tres primeras letras del nombre limpio en mayúsculas. Conserva los espacios interiores. Calcula el índice del último carácter a partir de la longitud.

Importante: Para coger la cadena entera, usar el metodo **nextLine()**, no useis el metodo next(), porque si meteis una frase con espacios entre medio, te va a coger solo hasta el siguiente espacio. Por ejemplo en " esfera azul " , te cogeria “   esfera “ solo, y queremos que te coja todo " esfera azul " (para eso usar nextLine().

Prueba 1: " esfera azul " → ESFERA AZUL; longitud 11; primero e; último l; prefijo ESF.

Prueba 2: "Sol" → SOL; longitud 3; primero S; último l; prefijo SOL. Las comillas delimitan el dato y no se introducen.

5 Una transformación que no se guarda

Intermedio · Comprender la inmutabilidad de String y aprovechar el retorno.

El programa debería conservar original y mostrar ORQUIDEA como etiqueta. Predice lo que muestra realmente. Los corchetes ayudan a ver los espacios.

Corrige las operaciones para crear limpio, mayusculas y etiqueta como tres variables diferentes. No cambies original. Usa las mismas transformaciones del fragmento y explica por qué las tres llamadas originales no lograban el resultado.

```
String original = "  orquidea roja  ";
original.trim();
original.toUpperCase();
original.substring(0, 8);
System.out.println("[" + original + "]");
```

Tras corregir: original mantiene los espacios; limpio vale "orquidea roja"; mayusculas vale "ORQUIDEA ROJA"; etiqueta vale "ORQUIDEA".

El original muestra [orquidea roja] con los espacios y todo

Métodos como trim(), toUpperCase() o substring() no modifican el objeto original, sino que calculan y devuelven un nuevo String con el resultado de la transformación.

original = " orquidea roja ";

```
String limpio = original.trim();
String mayusculas = limpio.toUpperCase();
String etiqueta = mayusculas.substring(0, 8);


System.out.println("original: [" + original + "]");
System.out.println("limpio: [" + limpio + "]");
System.out.println("mayusculas: [" + mayusculas + "]");
System.out.println("etiqueta: [" + etiqueta + "]");
```

6 Comparación de códigos de archivo

Intermedio · Comparar el contenido de cadenas y guardar resultados booleanos.

Lee dos códigos completos de archivo, cada uno en una línea. Quita los espacios de los extremos de ambos códigos y muestra dos boolean: si sus contenidos limpios coinciden exactamente y si coinciden después de convertir ambos a mayúsculas.

Usa equals para comparar contenido. No emplees == ni condiciones. Explica por qué la igualdad de referencias no responde a la pregunta planteada.

Prueba 1: " AB-12 " y "AB-12" → true y true.

Prueba 2: "ab-12" y "AB-12" → false y true. Prueba 3: "AB-12" y "AB-13" → false y false.

7 Registro de uso de un microscopio

Intermedio · Mezclar lecturas numéricas y de líneas completas con Scanner.

Lee, en este orden y en líneas distintas: número de puesto (int), nombre completo de la muestra (String) y aumento utilizado (double). Después muestra el puesto, el nombre sin espacios en los extremos y el aumento.

Configura Scanner con Locale.US para introducir el aumento con punto decimal. Resuelve el salto de línea que queda pendiente después del entero y explica para qué sirve la lectura adicional. Supón puesto y aumento positivos y un nombre no vacío.

Entrada: 12, " tejido vegetal " y 40.5, cada dato en una línea. Resultado: puesto 12, muestra tejido vegetal y aumento 40.5. El nombre no debe quedar vacío ni perder la segunda palabra.

8 Mediciones de una señal de sonido

Intermedio · Usar métodos static de Math y respetar sus retornos.

Lee dos amplitudes de una señal, de tipo double, que pueden ser positivas, negativas o cero. Obtén el valor absoluto de cada una y la mayor de esas dos magnitudes.

Calcula el cuadrado de la magnitud mayor con Math.pow, la raíz cuadrada de ese resultado con Math.sqrt y el entero más próximo a la magnitud mayor con Math.round. Muestra los seis resultados. Usa long para el resultado de round y Locale.US en la lectura.

Prueba 1: -3.5 y 2.0 → magnitudes 3.5 y 2.0; mayor 3.5; cuadrado 12.25; raíz 3.5; entero 4.

Prueba 2: -4.0 y -6.0 → 4.0; 6.0; 6.0; 36.0; 6.0; 6.

9 Elegir una firma de la API

Intermedio · Leer firmas y seleccionar variantes compatibles de una API.

Usa las firmas que aparecen debajo para preparar tres operaciones: obtener el máximo de 14 y 9, obtener el máximo de 14.2 y 9.8 y redondear 9.8 al entero más próximo. Para cada operación escribe una declaración Java con una variable compatible.

Anota la firma seleccionada, sus parámetros, los argumentos de tu llamada y el tipo de retorno. Indica el paquete de Math, si necesita importación explícita y si necesitas construir un objeto. Comprueba las tres llamadas con el autocompletado y ejecuta un programa que muestre los resultados.

```
// Firmas de la API para consultar, no para pegar en main:
public static int max(int a, int b)
public static double max(double a, double b)
public static long round(double a)
```

No conviertas los decimales a int antes de compararlos.

10 Asignación aleatoria de bandejas

Intermedio · Construir intervalos inclusivos con Random.nextInt.

Un taller de cerámica tiene bandejas numeradas del 20 al 35, ambos incluidos. Crea un único objeto Random y genera tres asignaciones independientes a ese intervalo. Usa tres llamadas explícitas, sin bucles, y muestra los resultados.

Explica cuántos enteros admite el intervalo, qué valores puede devolver nextInt antes del desplazamiento y por qué las bandejas pueden repetirse. No se pide que las tres asignaciones sean distintas.

Ejecuta el programa tres veces. Cada número debe estar entre 20 y 35. No hay una terna concreta obligatoria; puede repetirse un número dentro de una ejecución o entre ejecuciones.

11 Cinco errores en llamadas a métodos

Aplicación · Distinguir errores de compilación, ejecución y resultado.

Analiza cada caso de forma independiente, suponiendo String texto = "Archivo"; al inicio de cada uno. Clasifica el fallo como compilación, ejecución o resultado, explica la causa y escribe una corrección.

A: se quiere guardar la longitud. B: se quiere guardar el último carácter. C: se quiere conservar "Arch". D: se quiere mostrar ARCHIVO. E: se quiere guardar el entero más próximo a 4.7. No ejecutes los casos incorrectos como un único programa.

```
// A
int longitud = System.out.println(texto.length());
// B
char ultimo = texto.charAt(texto.length());
// C
String parte = texto.substring(0, 3);
// D
texto.toUpperCase();
System.out.println(texto);
// E
int entero = Math.round(4.7);
```

Después de corregir, los resultados deben ser 7, o, Arch, ARCHIVO y 5. En D puedes mostrar directamente el resultado de la transformación.

a)

int longitud = texto.length();

System.out.println(longitud);

B)

char ultimo = texto.charAt(texto.length() - 1);

C)

String parte = texto.substring(0, 4);

d(

String textoMayus = texto.toUpperCase();

System.out.println(textoMayus);

e)

int entero = (int) Math.round(4.7);

12 Informe de una muestra de germinación

Integración · Combinar Scanner, String, Math y Random en un programa lineal.

Lee, en este orden y en líneas distintas: número de muestra (int entre 100 y 999), nombre de la especie (String con al menos tres caracteres después de trim), altura anterior (double) y altura actual (double), en centímetros. Las alturas son cero o positivas. Usa Locale.US.

Prepara el nombre sin espacios de los extremos y en mayúsculas. Muestra también su longitud y un identificador compuesto por las tres primeras letras del nombre normalizado, un guion y el número de muestra.

Asigna una bandeja aleatoria del 4 al 12, ambos incluidos. Calcula cambio = altura actual - altura anterior y magnitud = valor absoluto del cambio. Redondea cambio a dos decimales con Math.round. Muestra número, nombre normalizado, longitud, identificador, bandeja, cambio redondeado y magnitud sin redondear.

Antes de programar, selecciona una operación de cada clase utilizada. Anota clase y paquete, constructor si lo hay, llamada, argumentos, carácter static o de instancia y tipo de retorno. Entrega esta ficha junto al programa.

Prueba 1: 314, " lavanda ", 2.5 y 5.75 → LAVANDA; longitud 7; LAV-314; cambio 3.25; magnitud 3.25.

Prueba 2: 208, " pino azul ", 8.25 y 7.0 → PINO AZUL; longitud 9; PIN-208; cambio -1.25; magnitud 1.25.

En ambas pruebas, bandeja de 4 a 12. Redondear el valor no obliga a imprimir dos cifras decimales; pueden aparecer pequeñas diferencias de representación en magnitud.